

Voice Pipeline Upgrade

Versi: 1.0 · **Tanggal:** 13 Juli 2026 · **Status:** Selesai & Terverifikasi · **Tipe:** Peningkatan arsitektur (Backend + iOS, Android di luar scope)

Dokumen ini melengkapi **PRD — Voice Assistant Mobile App** (baseline VC-1..VC-6) dan **Voice AI Assistant PRD** (spec generik yang dipakai sebagai checklist gap-analysis untuk upgrade ini).

1. Ringkasan

Voice pipeline Pip (Mode A: Gemini Live full-duplex; Mode B: VAD + STT + LLM + TTS custom dengan voice-cloning) sudah berjalan production sebelum upgrade ini, tapi diaudit ulang terhadap *Voice AI Assistant PRD* sebagai checklist arsitektur generik. Hasil audit: 4 gap nyata (state machine, provider abstraction, monitoring, noise suppression) plus satu item pembersihan (fitur greeting lama). Semua sudah diimplementasikan, diuji end-to-end dengan audio suara asli lewat kedua mode, dan diverifikasi bekerja.

2. Latar Belakang & Gap Analysis

Audit terhadap 5 area PRD, dibanding kode yang sudah ada:

Area PRD	Sebelum upgrade	Status
Voice State Machine	Flag ad-hoc (<code>_listening</code> / <code>_speaking</code> di Mode B); Mode A tanpa state sama sekali. Tidak ada yang dikirim ke client.	DIBANGUN
Voice Provider abstraction	<code>stt.py</code> / <code>tts.py</code> hardcode satu class (Whisper/XTTS), beda dengan pola config-driven <code>app/llm/</code> untuk LLM.	DIBANGUN
Monitoring	<code>app/llm/health.py</code> sudah lacak provider health/fallback count, tapi tidak pernah di-expose lewat endpoint apa pun. Latency per-turn cuma masuk log.	DIBANGUN
Noise Suppression	Tidak ada tahap denoising sebelum VAD/STT sama sekali.	DIBANGUN + BUG DITEMUKAN & DIPERBAIKI
Fitur greeting lama	Audio sambutan pre-recorded (cached) diputar otomatis begitu WS connect.	DIHAPUS

Echo cancellation **tidak** didobelkan di backend — itu sudah benar ditangani iOS lewat `AVAudioSession .voiceChat` mode (level OS, lebih tepat daripada software DSP di server). Android di luar scope karena tidak ada target Android di repo ini.

3. Diagram Arsitektur Pipeline



4. Voice State Machine

Modul baru `app/voice/state_machine.py`. State disatukan sebagai satu enum yang sama untuk kedua mode dan dikirim ke client lewat event WS baru: `{"event": "state", "state": "<nama>"}`.

```

IDLE —(user mulai bicara)→ LISTENING
LISTENING —(turn selesai / VAD end / PTT release)→ PROCESSING (Mode B saja – STT jalan)
PROCESSING —(STT selesai, teks kosong)→ IDLE
PROCESSING —(STT selesai, ada teks)→ THINKING (LLM men-generate, belum ada audio)
LISTENING —(Mode A: output pertama dari Gemini)→ THINKING
THINKING —(audio pertama mulai diputar)→ SPEAKING
SPEAKING —(user bicara lagi / barge-in)→ INTERRUPTED → LISTENING
* —(WS terputus)→ DISCONNECTED
* —(error tak tertangani)→ ERROR
Mode A saja: koneksi Gemini Live drop → RECONNECTING —(sukses)→ IDLE

```

State	Dipakai di	Arti
idle	A & B	Tidak ada turn aktif, siap menerima ucapan baru.
listening	A & B	Menangkap ucapan user.
processing	B	STT sedang jalan (Mode A tidak punya tahap ini terpisah — STT-nya inline di dalam API realtime Gemini).
thinking	A & B	LLM men-generate balasan, belum ada audio keluar.
speaking	A & B	Audio balasan sedang diputar.
interrupted	A & B	User memotong (barge-in) saat asisten bicara — VC-2/VC-3.
paused	tersedia, belum ada trigger nyata	Slot untuk fitur pause eksplisit di masa depan.
disconnected / error	A & B	Sesi berakhir tak normal.
reconnecting	A saja	Koneksi Gemini Live drop & sedang reconnect otomatis (kuirk API preview, sudah ada sebelum upgrade ini — sekarang terlihat oleh client).

iOS: `VoiceSocketClient.ServerState` (baru) menerima event ini dan menjadi *satu-satunya sumber kebenaran* untuk `VoiceSessionView.avatarState` — menggantikan logika lama yang merekonstruksi state dari kombinasi `phase` + `voice.isSpeaking` + `isInterruptedPulse`. Sistem animasi 5-state (`PipAvatarViewModel`, `breathing/glow`) tidak diubah — hanya cara *memetakan ke* sistem itu yang berubah.

5. Voice Provider Abstraction

Paket baru `app/voice/providers/`, meniru pola `app/llm/config.py` + `router.py` yang sudah ada untuk LLM: registry berbasis dict (bukan if/else), provider dipilih lewat file konfigurasi.

```

app/voice/providers/
  base.py          STTProvider / TTSProvider (Protocol)
  whisper_stt.py   WhisperSTTProvider → membungkus app/voice/stt.py (tidak ditulis ulang)
  xtts_tts.py      XTTSProvider       → membungkus app/voice/tts.py (tidak ditulis ulang)
  config.py        VoiceProviderConfig – baca config/voice_providers.yaml, fail-fast kalau ru
  factory.py       get_stt_provider() / get_tts_provider() – dict lookup, no if/else

```

`config/voice_providers.yaml`:

```

stt:
  default: whisper
  providers:
    whisper:
      model_size: small

tts:
  default: xtts
  providers:
    xtts:
      model_name: tts_models/multilingual/multi-dataset/xtts_v2

```

Hanya `whisper` / `xtts` yang jadi implementasi nyata saat ini — integrasi provider baru (OpenAI Realtime, ElevenLabs, Piper, Kokoro, dst. sesuai PRD) adalah kerja terpisah (butuh SDK/API masing-masing); fase ini baru membuat *pol*a-nya siap dipasang provider baru tanpa menyentuh `mode_b_pipeline.py` lagi — cukup tambah class + entry registry + blok config. Mode A (Gemini Live) tidak diikutkan ke abstraksi ini karena API-nya sudah menyatukan STT+LLM+TTS jadi satu — bukan sesuatu yang bisa dipecah ke slot STT/TTS terpisah.

6. Monitoring Endpoint

`GET /api/voice/metrics` (baru) — menggabungkan dua sumber yang sebelumnya tidak pernah keluar dari log/kode:

```

{
  "recent_turns": [
    {
      "turn_id": "1b0a2d57", "session_id": "...", "mode": "b",
      "stt_ms": 2231.4, "llm_first_token_ms": 474.2,
      "tts_first_chunk_ms": 16974.9, "total_ms": 19708.6,
      "at": "2026-07-13T09:04:21Z"
    }
  ],
  "llm_provider_health": { "gemini": "healthy", "groq": "healthy", ... },
  "llm_fallback_events_today": 22
}

```

`recent_turns` — ring buffer 50 turn terakhir (`app/voice/metrics.py`), diisi dari titik yang sama dengan log `LATENCY ...` yang sudah ada di `mode_b_pipeline.py` / `mode_a_gemini_live.py` . Mode A hanya punya satu

angka (`total_ms` = silence-wait + generation) karena STT-nya tidak terpisah dari API realtime-nya.
`llm_provider_health` / `llm_fallback_events_today` — data yang sudah dilacak `app/llm/health.py` sejak awal, sekarang baru pertama kali di-expose lewat endpoint.

7. Noise Suppression — Desain & Bug yang Ditemukan

Modul baru `app/voice/noise_suppression.py`, dipasang **terpusat** di `websocket_voice.py` 's `_client_frames()` supaya Mode A dan B dapat efeknya tanpa duplikasi kode.

Bug yang ditemukan saat verifikasi

Implementasi pertama menjalankan `noisereduce.reduce_noise(stationary=True)` langsung per-chunk mic (~100ms). Saat diuji dengan audio suara asli lewat WebSocket penuh, hasilnya:

Kondisi	Transkrip Whisper
Tanpa noise suppression	"halo pip, halo kamu bisa dengan suara saya halo pip pip kamu dengan suara saya halo"
Dengan noise suppression per-chunk 100ms (implementasi pertama)	<code>''</code> — kosong total

Spectral gating `stationary=True` butuh jendela cukup panjang untuk memperkirakan noise floor yang stabil — pada window 100ms yang isinya kebanyakan ucapan, algoritmanya salah menganggap sebagian besar sinyal sebagai "noise" dan menggatingnya habis. **Diperbaiki** dengan rolling buffer per-koneksi (~1 detik konteks, mengembalikan hanya bagian baru dari setiap panggilan) — hasil sama seperti tanpa denoising, dengan bug ini tidak lagi muncul:

Kondisi	Transkrip Whisper	Latensi rata-rata/chunk
Rolling window ~1s (implementasi final)	"halo pip, halo kamu bisa dengar suara saya halo pip pip, kamu dengar suara saya halo"	~5ms (maks ~10ms)

Ada kill-switch `voice_noise_suppression_enabled` (default `True`) di `app/config.py` kalau suatu saat perlu dimatikan tanpa ubah kode — budget latency PRD untuk streaming delay (<100ms) cukup ketat.

8. Fitur Greeting Lama — Dihapus

`app/voice/greetings.py` dan `scripts/generate_greetings.py` dihapus, beserta cache audio 2.4MB yang sudah tidak terpakai (`data/greetings/`). Blok pemutaran greeting di `websocket_voice.py` dihapus. Tidak ada perubahan di sisi iOS — client tidak punya logika khusus greeting, hanya merender event `reply` /audio apa pun yang datang.

9. Hasil Verifikasi End-to-End

Diuji dengan audio suara asli (bukan sintetis) dikirim lewat koneksi WebSocket nyata ke kedua mode:

Mode	Urutan state	Transkrip	Balasan LLM	Audio balasan
B (Whisper→LLM→XTTS)	listening → processing → thinking → speaking → idle	"halo pip, halo kamu bisa dengar suara saya halo, pip pip, kamu dengar suara saya halo"	"Gue denger suara lo, bro! Lagi apa sih sekarang? ..."	16 frame, 636KB PCM16
A (Gemini Live)	listening → thinking → speaking → idle	(transkrip Gemini sendiri, realtime)	"Halo Rakka! Bisa, dengar kok suaranya. Ada yang bisa kubantu?"	80 frame, 197KB PCM16

Build iOS (`xcodebuild`) sukses setelah seluruh perubahan Swift (state consumption).

10. Referensi File

File	Peran
<code>backend/app/voice/state_machine.py</code>	Baru — <code>VoiceState</code> enum + <code>VoiceStateMachine</code> .
<code>backend/app/voice/mode_b_pipeline.py</code>	Diubah — state machine, factory provider, metrics.
<code>backend/app/voice/mode_a_gemini_live.py</code>	Diubah — state machine, metrics.
<code>backend/app/api/websocket_voice.py</code>	Diubah — greeting dihapus, noise suppression terpusat.
<code>backend/app/voice/providers/</code>	Baru — base/whisper_stt/xtts_tts/config/factory.
<code>backend/config/voice_providers.yaml</code>	Baru — konfigurasi provider STT/TTS.
<code>backend/app/voice/metrics.py</code>	Baru — ring buffer turn latency.
<code>backend/app/api/monitoring.py</code>	Baru — <code>GET /api/voice/metrics</code> .
<code>backend/app/voice/noise_suppression.py</code>	Baru — <code>NoiseSuppressor</code> rolling-window.
<code>backend/app/config.py</code>	Diubah — flag <code>voice_noise_suppression_enabled</code> .
<code>Pip/Core/Networking/VoiceSocketClient.swift</code>	Diubah — <code>ServerState</code> enum + event baru.
<code>Pip/Features/Main/VoiceSessionView.swift</code>	Diubah — <code>avatarState</code> pakai <code>serverState</code> .
<code>backend/app/voice/greetings.py</code> , <code>scripts/generate_greetings.py</code>	Dihapus.